SECURELOG

Prepared by Amogh Kulkarni - 23102C0003

INTRODUCTION

- Security Operations Centers (SOC) face continuous high-velocity intrusion logs (SSH brute force, SQL injections) that overwhelm traditional manual review workflows.
- Traditional log management relies on imperative shell scripts, lacks automated browser acceptance testing, and suffers from severe environment drift between staging and production.
- **SecureLog** resolves this by implementing a complete 14-stage DevSecOps pipeline: combining Spring Boot 3, automated **Jenkins Continuous Testing Quality Gates**, **hardened non-root Docker containerization**, and **Ansible idempotent provisioning**.

COMPARATIVE SYSTEM ANALYSIS

Approach	Technique	Characteristics	Result
Traditional Method	Manual Shell Scripts	Imperative deployment commands; host-level configuration drift	Slow, prone to human error; lacks repeatability and idempotency
Basic CI Pipeline	Unit Tests Only	Tests isolated code logic; no automated UI acceptance validation	High UI bug leak rate; zero end-to-end regression proof
Standard Docker	Default Root Container	Runs as root (UID 0); bloated base images (>600 MB)	High CVE attack surface; privilege escalation vulnerability
SecureLog (Proposed)	Jenkins + Docker + Ansible	Headless Selenium Quality Gate, 166 MB Alpine JRE, Non-root UID 1001, Ansible (changed=0)	Automated Quality Gates, <15s rollback, 100% environment parity, 0 drift

METHODOLOGY

- **1. Ingestion & MVC:** Spring Boot 3 + in-memory H2 JPA ingest threats with real-time severity classification (CRITICAL, HIGH, LOW).
- **2. GitFlow Architecture:** Strict branch isolation across `dev` and `feature/*` with annotated release tags (`v1.0.0`, `v1.1.0`).
- **3. Continuous Testing Quality Gate:** Headless Chrome Selenium suite validates 5 user journeys inside `Jenkinsfile`.
- **4. Idempotent Provisioning:** Modular Ansible roles deploy containers with verified idempotency (`changed=0`).

OBJECTIVES OF THE PROJECT

- To implement a fully automated CI/CD pipeline enforcing automated Quality Gates, preventing defective code from reaching staging.
- To package the application into a minimal, secure OCI container running under an unprivileged user (UID 1001).
- To establish Infrastructure-as-Code via Ansible with proven mathematical idempotency and automated zero-downtime rollback.
- To maintain 100% audit compliance and evidence across all 14 curriculum DevOps milestones.

SUMMARY OF FINDINGS

- **Continuous Testing Quality Gate:** Deliberately injected defect halted pipeline and captured failure screenshot; defect resolved and achieved 100% green pass in **27.13 seconds**.
- **Ultra-Compact Footprint:** Multi-stage Alpine container optimized image to only **166 MB** with non-root security context (`appuser:1001`).

- **Proven Idempotency:** Re-running Ansible playbook achieved strict mathematical zero drift (**`ok=10 changed=0 failed=0`**).
- **Rapid Disaster Recovery:** Automated rollback drill (`rollback.yml`) restored healthy production state in **under 15 seconds (RTO < 15s)**.

CONCLUSION

- **SecureLog** proves that combining automated Continuous Testing Quality Gates with declarative containerization and Ansible configuration management transforms manual, error-prone software delivery into a secure, verifiable, and resilient enterprise release pipeline. By enforcing non-root container isolation and proven idempotency, the system guarantees 100% environment parity and rapid disaster recovery ready for production SOC operations.

Faculty Sign- _____ **External Examiner's Signature –** _____